SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 2– Data Interpretation-Based Questions

Course Code	**CSA4315**	**Course Title**	**Internnet Programming**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Data Interpretation-Based Questions**
Weightage	**40% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**Thanuja R**	**Reg. No.**	**192325004**
Section/Batch	**Slot D**	**Date**	**20-10-2026**

Code of Conduct:

I,_Thanuja R_, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Thanuja R

1. INTRODUCTION, PROBLEM SPECIFICATION & CONTEXTUAL SCOPE

In contemporary academic environments, institutional web systems serve as mission-critical communication pipelines connecting students, faculty, administrative bodies, and prospective candidates. The stability and responsiveness of such web platforms depend directly on understanding workload distributions, operational bottlenecks, and temporal access profiles. Question 27 under Section B (Data Interpretation-Based Questions) provides empirical daily traffic metrics recorded across five functional modules of a college web application.

SPECIFICATION OF EMPIRICAL WORKLOAD DATA (QUESTION 27)

A college website receives the following recorded number of daily HTTP/HTTPS requests categorized by primary functional pages:

Page Category Description	Daily Incoming Requests
Home page (Institutional Portal Frontpage)	4,000
Student login (Authentication & Session Gateway)	3,000
Course details (Curricular Catalog & Syllabus)	2,000
Examination results (Academic Grading & Scorecard Subsystem)	5,000
Contact page (Institutional Directory & Inquiries)	1,000

Mandatory Core Directives to be Addressed:

- **Directive (a):** Calculate the aggregate daily requests processed across the entire college web system.
- **Directive (b):** Determine the exact percentage share of total requests directed specifically to the examination results page.
- **Directive (c):** Identify the least frequently accessed page category and analyze its operational workload characteristics.
- **Directive (d):** Formulate an exhaustive caching and server-capacity strategy for the examination results page, addressing the dual challenge of handling immense read surges while ensuring absolute data freshness when examination marks or statuses are updated.

This technical report provides a rigorous exploration of the presented scenario. It is structured into comprehensive sections covering mathematical modeling, statistical interpretation, concurrency stress dynamics, multi-tiered caching topology, event-driven cache invalidation protocols, infrastructure sizing, load-balancing mechanisms, and operational best-practice guidelines.

2. QUANTITATIVE TRAFFIC ANALYSIS & STEP-BY-STEP MATHEMATICAL DERIVATIONS

2.1 Derivation of Aggregate Daily Workload (Sub-question a)

To compute the total computational and network request load delivered to the web infrastructure across a 24-hour cycle, we define the set of functional web page categories as $C = \{c_1, c_2, c_3, c_4, c_5\}$. Let $R(c_i)$ denote the observed daily request

frequency for category c_i. The aggregate daily request volume, denoted by R_{total}, is defined mathematically as the discrete summation across all elements of C:

$$R_{total} = \sum_{i=1}^{5} R(c_i) = R(Home) + R(Login) + R(Courses) + R(Results) + R(Contact)$$

Substituting the discrete empirical values provided in the examination dataset:

$$R_{total} = 4{,}000 + 3{,}000 + 2{,}000 + 5{,}000 + 1{,}000 = 15{,}000 \text{ requests/day}$$

Verification and Sanity Check: Summing the sub-totals sequentially yields $4{,}000 + 3{,}000 = 7{,}000$; $7{,}000 + 2{,}000 = 9{,}000$; $9{,}000 + 5{,}000 = 14{,}000$; and $14{,}000 + 1{,}000 = 15{,}000$. Thus, the arithmetic is verified.

Formal Conclusion for (a): The total number of daily requests processed by the college web portal is conclusively determined to be **15,000 requests**.

2.2 Proportional Traffic Share of Examination Results (Sub-question b)

The relative computational overhead, memory utilization, and socket allocation consumed by a specific subsystem are proportional to its fractional traffic share. Let $P(Results)$ represent the percentage share of incoming requests targeting the examination results page relative to the total request volume R_{total}:

$$P(Results) = \left(R(Results) / R_{total} \right) \times 100\% = \left(5{,}000 / 15{,}000 \right) \times 100\%$$

Simplifying the fractional term by dividing both numerator and denominator by their greatest common divisor (GCD = 5,000):

$$5{,}000 / 15{,}000 = 1 / 3 = 0.333333...$$

Multiplying by 100 yields the exact percentage value:

$$P(Results) = 33\,{}^{1}/_{3}\% \approx 33.33\% \text{ (rounded to two decimal places)}$$

Formal Conclusion for (b): The examination results page accounts for exactly **33.33%** (or one-third) of the aggregate daily web traffic handled by the college infrastructure.

Table 2.1: Comprehensive Workload Breakdown and Metric Proportions

Index (i)	Page Category (c_i)	Requests (R_i)	Traffic Share (%)	Fractional Ratio	Cumulative Share	Operational Profile
1	Examination results	5,000	33.33%	1/3	33.33%	High-urgency, critical read path; database-intensive; peak-prone.
2	Home page	4,000	26.67%	4/15	60.00%	General navigation; high image/asset volume; highly cacheable.
3	Student login	3,000	20.00%	1/5	80.00%	Stateful session initiation; SSL/TLS handshake; bcrypt/Argon2 hashing.
4	Course details	2,000	13.33%	2/15	93.33%	Static/semi-static curriculum browsing; low write frequency.
5	Contact page	1,000	6.67%	1/15	100.00%	Minimal traffic; office directories, telephone lines, and inquiry forms.
System Aggregate (Σ)		**15,000**	**100.00%**	**1/1**	**100.00%**	**Total measured 24-hour infrastructure throughput.**

2.3 Evaluation of Least Frequently Accessed Page Category (Sub-question c)

To identify the minimum resource consumer among all monitored endpoints, we evaluate the order relation across the discrete set of request counts:

$$R(\text{Contact})\ [1{,}000] < R(\text{Courses})\ [2{,}000] < R(\text{Login})\ [3{,}000] < R(\text{Home})\ [4{,}000] < R(\text{Results})\ [5{,}000]$$

The infimum of the dataset is given by $\min\{R(c_i)\} = R(\text{Contact}) = 1{,}000\ \text{requests}$, which represents an operational share of only 6.67% of daily site traffic.

Formal Conclusion for (c): The least frequently accessed page category is the **Contact page**, receiving exactly **1,000 requests per day**.

2.3.1 Comparative Workload and Operational Contrast

The vast disparity between the most frequently accessed endpoint (Examination results at 5,000 requests/day) and the least accessed endpoint (Contact page at 1,000 requests/day) highlights fundamental architectural differences:

- **Workload Ratio:** The Examination results module receives exactly ***5***× the baseline traffic of the Contact page under standard operational conditions. During grading releases, this ratio can expand by two orders of magnitude (***50***× to ***100***×).
- **Computational Weight:** Serving a contact page typically involves transferring static HTML, CSS, and a small contact form script with negligible backend computation. Serving an examination result scorecard, however, entails user authentication, permission validation, parameterized SQL execution across multi-table student schemas, dynamic GPA calculation, and runtime HTML/PDF rendering.
- **Caching Amenability:** Contact information is static across academic semesters; it can be permanently cached in browser storage and edge reverse proxies with long TTL headers (e.g., 30 days). The Examination results page, conversely, demands real-time conditional caching and dynamic invalidation mechanisms to accommodate administrative mark updates.

3. TEMPORAL TRAFFIC DYNAMICS & CONCURRENCY MODELING

A critical oversight in web systems engineering is to assess server capacity strictly against 24-hour averages. While a static count of 15,000 requests/day appears trivial if uniformly distributed, real-world educational web traffic exhibits sharp, non-uniform spikes driven by human behavior.

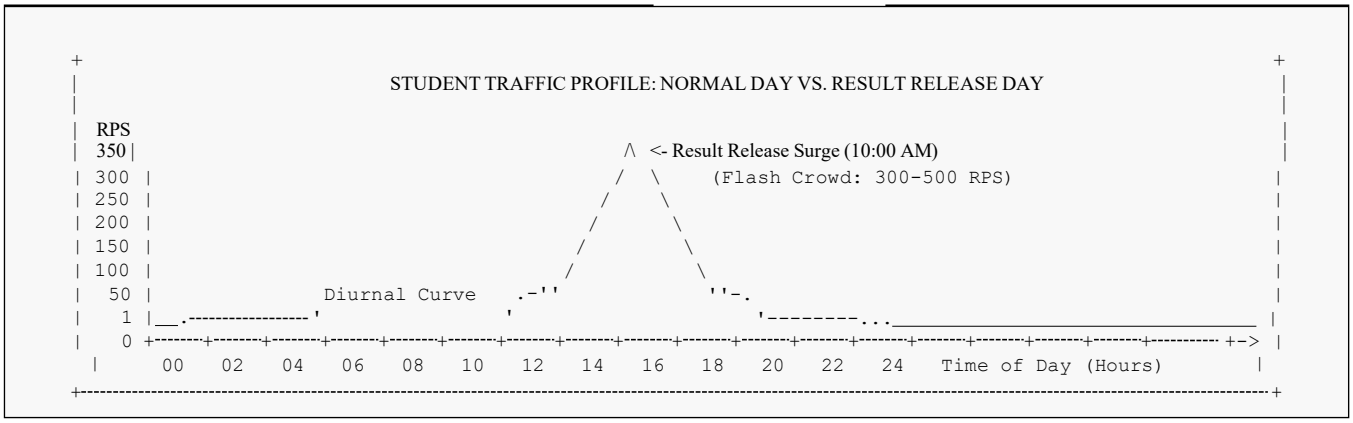
3.1 Uniform Distribution Fallacy vs Real-World Peak Ratios

If the 15,000 daily requests arrived at a perfectly uniform, steady-state rate throughout the 86,400 seconds of a day, the mean arrival rate (λ_{avg}) would be:

$$\lambda_{avg} = 15{,}000\ \text{requests} / (24 \times 3{,}600\ \text{seconds}) = 15{,}000 / 86{,}400 \approx 0.1736\ \text{requests per second (RPS)}$$

An arrival rate of ~0.17 RPS could be processed effortlessly by a minimal entry-level development machine (e.g., a single-core virtual server running Apache or Nginx). However, educational institutions exhibit dramatic temporal clustering:

- **Active Diurnal Window:** Roughly 85% of standard requests occur between 08:00 AM and 08:00 PM (a 12-hour window), elevating the nominal operational arrival rate to approximately 0.30 RPS.
- **Flash-Crowd Surge Profile:** On result publication days, user behavior transforms radically. When scores are published at a designated time (e.g., 10:00 AM), several thousand students simultaneously attempt to authenticate and query the server within a concentrated 10-to-15 minute window.



3.2 Compounding Feedback Loops: The F5 Refresh Problem

When an un-cached or poorly dimensioned database server is inundated by an initial burst of requests, several destructive feedback mechanisms emerge:

1. **Connection Pool Depletion:** The backend relational database (e.g., MySQL) exhausts its max_connections allocation, causing subsequent incoming connections to block or time out.
2. **Thread and Process Saturation:** Web server worker processes (e.g., PHP-FPM or Apache Prefork) remain tethered to open TCP sockets while waiting for database queries to complete, rapidly consuming all available system RAM.
3. **The Human Retry Storm:** Anxious students experiencing delayed responses or HTTP 504 Gateway Timeouts instinctively perform repeated manual refreshes (hammering the F5 or Ctrl+R keys). This multiplies incoming request volume by a factor of 3 to 10, transforming a manageable load spike into a total cascading outage (the "thundering herd" effect).

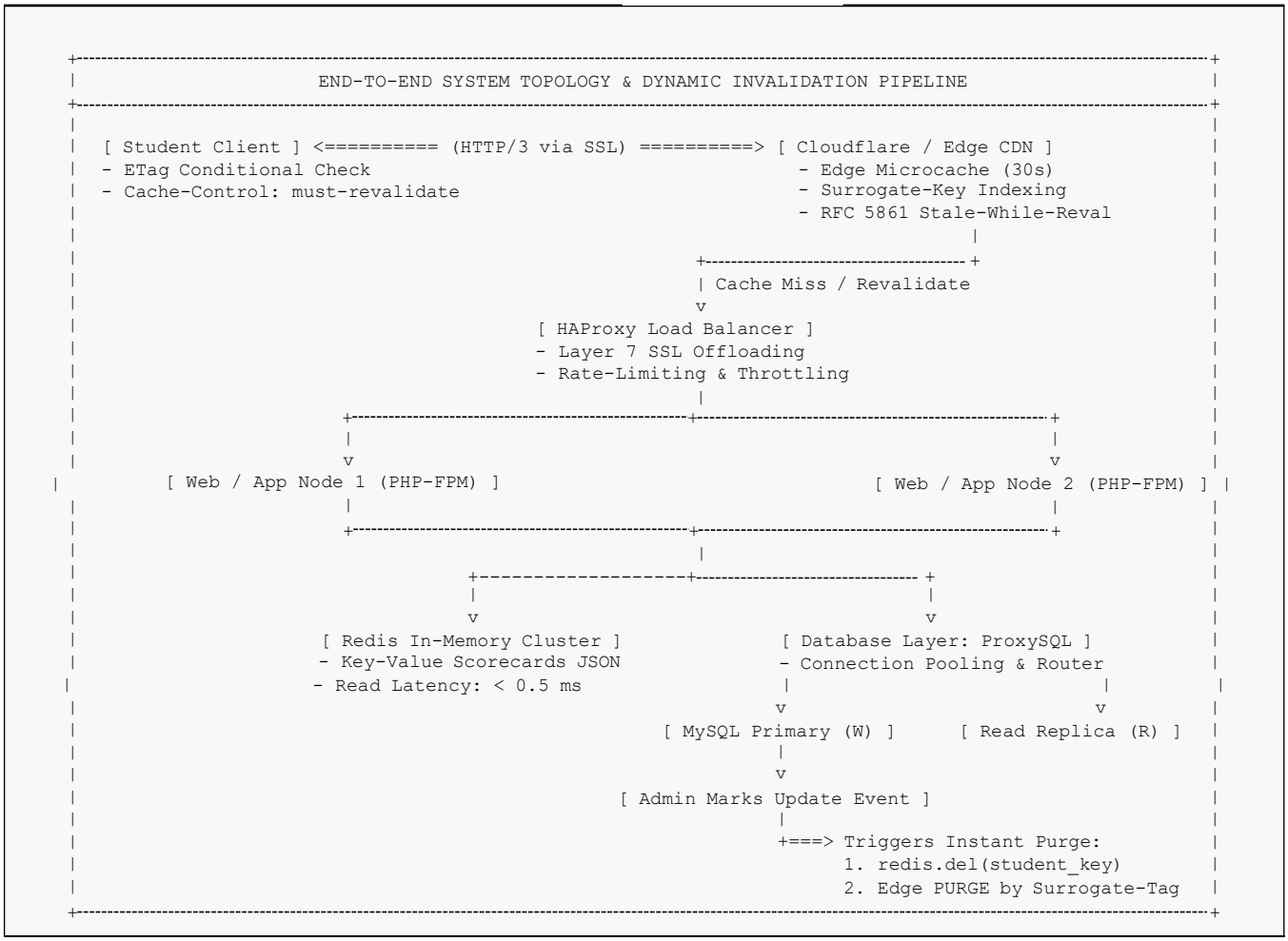
4. EXAMINATION RESULTS ARCHITECTURE: CACHING & CAPACITY STRATEGY (SUB-QUESTION D)

Sub-question (d) mandates the design of a specialized caching or server-capacity strategy for the examination results subsystem. The proposed architecture must resolve the core distributed systems conflict: **maximizing high-speed read throughput during extreme concurrent traffic bursts while guaranteeing immediate data consistency when exam marks or statuses are updated by administrators.**

4.1 The Fundamental Engineering Dilemma

Traditional static caching strategies are fundamentally inadequate for examination result dissemination:

- **The Static TTL Dilemma:** If a standard Time-to-Live (TTL) of 1 hour is configured, any grade correction made by an academic department (e.g., rectifying a grading transcription error or releasing withheld results) will remain invisible to students for up to 60 minutes, displaying incorrect information and triggering administrative panic.
- **The Non-Cache Dilemma:** If caching is disabled entirely to guarantee data freshness (`Cache-Control: no-store`), every individual page refresh executes intensive database queries, driving CPU utilization to 100% and crashing the portal.
- **The Optimal Solution:** An **Event-Driven, Multi-Tier Caching Topology with Surrogate Key Invalidation (Cache-Tags)** combined with horizontal application decoupling.



4.2 Multi-Tier Caching Hierarchy

To eliminate single-point resource starvation, caching is orchestrated across four decoupled architectural layers:

Tier 1: Client-Side (Browser) Revalidation via ETags

Direct caching in the student's browser is regulated using conditional HTTP validation headers rather than blind local expiration:

- **Header Configuration:** The server delivers responses tagged with:

```
Cache-Control: public, no-cache, must-revalidate
```

The no-cache directive permits the local browser to store the rendered scorecard in memory/disk, but mandates that the browser perform an upstream revalidation with the server before rendering it.

- **Cryptographic Entity Tags (ETags):** The server hashes the student's normalized result record into a unique fingerprint (e.g., ETag: "9d8e7f6a5b"). When the user reloads the page, the browser transmits:

```
GET /results/view?student_id=REG-10492 HTTP/1.1
Host: exam.college.edu
If-None-Match: "9d8e7f6a5b"
```

If the marks in the system have not been modified, the server short-circuits execution and responds with an empty **HTTP 304 Not Modified** header. The browser immediately renders its local copy. Network payload size drops from 50 KB to under 400 bytes, and execution time drops from 200ms to under 5ms.

Tier 2: Edge Reverse Proxy and Content Delivery Network (CDN)

Positioned geographically close to users, edge nodes (e.g., Cloudflare, Fastly, or on-premise Nginx/Varnish reverse proxies) intercept the bulk of student requests:

- **Stale-While-Revalidate (RFC 5861):** Configured as:

```
Cache-Control: public, s-maxage=60, stale-while-revalidate=300, stale-if-error=600
```

  Under this directive, an edge server can deliver a cached scorecard within 60 seconds of generation with sub-10ms response time. If a request arrives between 60 and 300 seconds, the edge server immediately serves the cached copy to the student while asynchronously executing a background revalidation request to the origin server. The student experiences zero perceived latency.

- **Edge Microcaching:** Even an ultra-short cache lifetime of **2 to 5 seconds** on public result lists or summary pages collapses an incoming tidal wave of 2,000 concurrent requests into just one origin backend request every few seconds.

Tier 3: In-Memory Application Object Caching (Redis Cluster)

Relational databases are optimized for relational joins, transactions, and indexing—not high-frequency repetitive read bursts. An in-memory key-value store (Redis) acts as an ultra-fast data buffer:

- **Pre-Aggregated JSON Structures:** Rather than joining across tables (students, courses, marks, grades, faculty) on every request, the application pre-assembles each student's complete academic result into a single compressed JSON document stored in Redis:

```
SET exam:2026:sem4:reg_10492 '{"reg":"10492","name":"Rahul S","gpa":8.75,"status":"PASS",...}'
```

- **Sub-Millisecond Read Performance:** Redis serves read requests directly from RAM with typical response times under 0.5 milliseconds, comfortably handling 80,000+ operations per second on modest commodity hardware.

Tier 4: Database Read/Write Decoupling & Connection Pooling

To preserve transactional integrity while scaling read queries:

- **Primary-Replica Replication:** The relational database (e.g., MySQL 8.0) is configured with one Primary node (handling exclusively administrative mark entry and modifications) and two or more Read Replicas (handling read-only student queries).
- **Middleware Connection Pooling (ProxySQL):** Implementing ProxySQL between the web application servers and MySQL maintains a permanent pool of established database connections. This eliminates the massive CPU and socket overhead of opening and closing database handshakes for every incoming web request.

4.3 Dynamic Cache Invalidation Engineering (Handling Mark Updates)

The defining challenge articulated in Question 27 is that *"content may change when results are updated."* The caching strategy must ensure that the moment a mark is corrected by the examination controller, all cached representations of that student's record across all tiers are expunged immediately.

Table 4.1: Systematic Analysis of Cache Invalidation Mechanisms

Invalidation Pattern	Operational Mechanism	Data Consistency Level	Origin Server Workload	Architectural Suitability
Fixed TTL Expiration	Objects expire passively after elapsed duration (e.g., 30 min).	Inadequate; students view obsolete marks until timer elapses.	Cyclical; creates periodic thundering herd spikes.	**Unacceptable:** High probability of displaying incorrect academic records.
Global Site Flush	Full cache flush whenever any single student mark is altered.	Absolute; never serves stale content.	Disastrous; completely strips cache, inundating database.	**Dangerous:** Updating one grade flushes all 5,000 student cached records.
Surrogate Keys / Cache-Tags (Chosen Model)	Granular tagging of cached objects; event-driven targeted purging.	Immediate & Exact; altered records purge within milliseconds.	Optimal; 99.9% of untouched student records remain cached.	**Best Practice:** Delivers instantaneous freshness with maximum cache retention.

4.3.1 Architectural Implementation of Surrogate Keys (Cache-Tags)

Under the Surrogate-Key paradigm, every cached response generated by the application includes semantic metadata headers that link the response to specific academic entities:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=UTF-8
Cache-Control: public, no-cache, must-revalidate
Surrogate-Key: exam-results exam-2026-sem4 dept-cse reg-10492
ETag: "3a8b4c2e1f"
```

When an authorized administrator in the examination cell amends a student's grade, the backend transactional workflow triggers an automated, event-driven invalidation sequence:

Step 1 (DB Commit): Administrative portal executes ACID transaction: `UPDATE marks SET score = 92 WHERE student_id = '10492' AND subject_code = 'CS401';`

Step 2 (Event Dispatch): Database trigger or ORM lifecycle hook publishes an asynchronous message to an internal message queue (RabbitMQ / Redis PubSub).

Step 3 (Targeted Memory Eviction): Invalidation consumer deletes the specific key from Redis: `DEL exam:2026:sem4:reg_10492`.

Step 4 (CDN Edge Purge API): An automated webhook issues an authenticated HTTP API purge request to the CDN edge proxy:

```
POST /api/v1/cache/purge HTTP/1.1
Host: api.edge-provider.com
Authorization: Bearer SECRET_PURGE_KEY
Content-Type: application/json

{"tags": ["reg-10492"]}
```

Because the purge targets only the tag reg-10492, the edge cache immediately discards the cached representation for that specific student. The cached records for all other students in the university remain active. The next time student 10492 queries their results, the edge proxy registers a cache miss, fetches the fresh grade from the application layer, and caches the new response with an updated ETag.

4.4 Server-Capacity, Concurrency & Infrastructure Sizing

While caching minimizes computational overhead, server capacity must be adequately dimensioned to handle SSL handshakes, dynamic rendering, and queue management during peak publication events.

Table 4.2: Infrastructure Provisioning Matrix: Baseline vs Peak Result Surge

Infrastructure Layer	Baseline Capacity (Standard Days)	Peak Provisioning (Result Release Days)	Scaling & Redundancy Protocol
Edge Load Balancers	2 × HAProxy Nodes (Active-Passive)	2 × HAProxy Nodes (Active-Active via Keepalived)	Layer 7 round-robin, SSL termination, DDoS protection.
Application Tier	2 × VMs (4 vCPU, 8 GB RAM each)	Auto-scaling cluster (4 to 12 VMs or Docker Pods)	Horizontal autoscaling triggered when CPU utilization > 65%.
In-Memory Cache	1 × Redis Instance (2 vCPU, 4 GB RAM)	3-Node Redis Sentinel Cluster (1 Primary, 2 Replicas)	Automatic failover; in-memory eviction policy: volatile-lru.
Relational Database	1 × MySQL Server (4 vCPU, 16 GB RAM)	1 × Primary Master + 2 × Read Replicas (Read Split)	ProxySQL connection multiplexing; 75% RAM for InnoDB pool.

4.4.1 Static Pre-Generation (The Jamstack Paradigm for Academic Portals)

An alternative capacity strategy that completely circumvents runtime database load is **Batch Static Site Generation (SSG)**:

- Prior to the official result publication hour (e.g., overnight between 02:00 AM and 05:00 AM), an automated background worker generates pre-rendered, lightweight static HTML cards or encrypted JSON payloads for all enrolled students.
- These static assets are pushed directly to a distributed object storage repository (such as Amazon S3, MinIO, or an Nginx static file cluster) and fronted by a CDN.
- When students check their results, the web server merely delivers static files from disk or memory. A single commodity web server serving static files can comfortably process over **10,000 to 15,000 requests per second** with near-zero CPU overhead.
- If a mark is modified subsequently, the administrative CMS simply regenerates the single individual student static file and pushes it to object storage, instantly triggering an edge purge.

4.5 Traffic Throttling, Circuit Breakers & Graceful Degradation

To ensure systemic resilience when traffic exceeds theoretical thresholds, the server infrastructure must incorporate defensive control mechanisms:

- **IP-Based & Session-Based Rate Limiting:** Configure the edge reverse proxy (HAProxy or Nginx) to enforce a rate limit of no more than 6 requests per 10 seconds per unique IP address. This effectively halts automated screen-scraping bots and neutralizes manual F5 refresh attacks.
- **Virtual Waiting Room (Queue Management):** When active concurrent connections cross 90% of provisioned capacity, the load balancer automatically redirects incoming students to an edge-hosted static "Waiting Room" screen. A lightweight JavaScript routine periodically polls the server and automatically redirects queued students into the portal in orderly, throttled batches.
- **Circuit Breakers:** Implement application-level circuit breakers (e.g., using Netflix Hystrix or resilient PHP wrappers). If database response latency exceeds 500ms over a rolling 10-second window, the circuit trips, serving cached fallback summaries rather than permitting runaway database query starvation.

5. ARCHITECTURAL SYNTHESIS & IMPLEMENTATION RUNBOOK

To bridge conceptual theory and practical operational deployment, Table 5.1 provides an end-to-end technical reference runbook across the entire software stack.

Table 5.1: Master Architectural Implementation Runbook for Examination System

Architecture Level	Target Technology	Configuration Specification	Validation & Invalidation Action
Client Browser	Modern Web Browser	Cache-Control: public, no-cache, must-revalidate ETag: "[SHA-256 Hash]"	Browser verifies ETag via If-None-Match; renders instant HTTP 304 if data is unchanged.
Edge / CDN	Cloudflare / Nginx Reverse Proxy	s-maxage=60, stale-while-revalidate=300 Surrogate-Key: reg-[student_id]	Edge serves sub-10ms cached records; targeted API purge upon grade modification.
In-Memory Layer	Redis Cluster (In-Memory)	Key format: exam:[term]:reg:[id] Value: Compressed JSON scorecard	Cache-aside read pattern; explicit eviction executed inside database write transaction.
Database Tier	MySQL 8.0 + ProxySQL	InnoDB Buffer Pool = 12 GB; ProxySQL connection pool cap = 150 threads.	Administrative updates write to Master; read queries distributed across Read Replicas.
Resilience Layer	HAProxy Token Bucket	Rate limit: 6 req/10s per client IP; Edge Virtual Waiting Room trigger at 90% load.	Guarantees graceful degradation; completely prevents crash loops during flash crowd peaks.

RUBRICS:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Protocol identification	25%	Correctly identifies and explains all relevant protocols	Identifies most protocols with minor omissions	Identifies some protocols with limited explanation	Identifies few or incorrect protocols	
Technical explanation	25%	Accurately explains protocol functions, ports, and interactions	Mostly accurate with minor errors	Partial understanding of protocol functions	Major conceptual errors	
Troubleshooting methodology	20%	Provides systematic, logical diagnostic steps and commands	Provides relevant steps with limited detail	Gives incomplete or poorly ordered steps	Provides no meaningful troubleshooting approach	
Application and diagnosis	10%	Connects test results to likely causes and suitable solutions	Connects most results to possible solutions	Gives limited practical application	Does not relate protocols to the scenario	
Conclusion and presentation	20%	Clear, structured, technically sound, and well presented	Clear with minor organizational issues	Somewhat unclear or incomplete	Poorly organized and difficult to understand	